PROJECT GUIDE / FULL TECHNICAL WALKTHROUGH

DeepSignal: Hybrid Stock Trend Prediction - Complete Project and Proposal Guide

A detailed, plain-language guide to the application, its architecture, its model pipeline, how it compares with the supplied proposal, and how to run it on another computer.

Prepared: 20 October 2026
Project type: Local full-stack research prototype
Main technologies: React, Vite, FastAPI, Python, pandas, scikit-learn, TensorFlow/Keras, Hugging Face Transformers
Source documents: current project source code and Improved_Hybrid_Stock_Prediction_Proposal.pdf

A plain-language guide to the code as it is implemented today.

Contents

Use this map to jump to any part of the guide.

01 01. Executive summary 3

02 02. What the application does 4

03 03. System architecture 5

04 04. Project files and responsibilities 6

05 05. Market data and news 7

06 06. Feature engineering and indicators 8

07 07. The model components 9

08 08. How the final signal is calculated 11

09 09. Evaluation and model comparison 12

10 10. Backtesting and its limits 13

11 11. API reference 14

12 12. Frontend behavior and local storage 15

13 13. Tuning parameters 16

14 14. Installation on another computer 17

15 15. Running the validation suite 19

16 16. Troubleshooting 20

17 17. Security, privacy, and operating boundaries 21

18 18. Current limitations and sensible next steps 22

19 19. Glossary 23

20 20. Final perspective 24

21 21. How this guide uses the proposal 25

22 22. Deep dive: LSTM model 27

23 23. Deep dive: Random Forest 28

24 24. Deep dive: XGBoost and model comparison 29

25 25. Model fusion and signal interpretation 30

26 26. Architecture and data flow in greater detail 31

27 27. Evaluation gaps and honest interpretation 32

28 28. Setup, reproducibility, and delivery checklist 33

01. Executive summary

DeepSignal is a local web application for exploring recent stock data and producing an experimental directional signal. A user enters a stock ticker in the Analysis page. The browser sends a request to a Python API. The API downloads recent market history, calculates technical indicators, looks for financial news, optionally scores news with FinBERT, runs the LSTM and Random Forest components, blends the evidence, and returns a response. The browser displays the result, charts, news headlines, indicators, and the factors behind the signal.

The application is designed as a research and learning project. Its output is an estimate generated from historical data and simplified rules. It is not a brokerage system, does not place trades, and does not guarantee future returns. A Buy label does not mean a stock will rise, and a Hold label does not mean its price will remain unchanged.

The main parts are:

- A React frontend with Dashboard, Analysis, Backtest, Models, Tuning, Signals, Research, and Settings pages.
- A FastAPI backend that exposes JSON endpoints to the frontend.
- Yahoo Finance history and news as the primary data sources, with optional NewsAPI integration.
- Technical indicators calculated from OHLCV history.
- An LSTM binary classifier, a Random Forest classifier, optional XGBoost comparison, and optional FinBERT sentiment.
- A weighted decision rule that converts model evidence into Buy, Sell, or Hold.
- A separate rule-based backtest, a browser-local signal history, and persisted tuning settings.

02. What the application does

Dashboard

The Dashboard shows summary cards for saved signals and a selectable market chart. The chart selector includes Apple by default and tickers found in locally saved signals. Selecting a ticker requests three months of daily market history from the backend. It then displays a candlestick chart, latest price, period change, RSI, MACD, average volume, and a simple price-momentum gauge.

The Dashboard's Buy/Sell counts and latest signals come from predictions saved in the current browser. They are not a shared account-wide database. Opening the site in another browser or clearing browser storage will not carry those saved signals over.

Analysis

The Analysis page accepts a ticker and one of four horizons: 1D, 1W, 1M, or 3M. It requests a prediction and displays the latest available close and date, signal badge, trend, heuristic predicted change, FinBERT sentiment score, confidence-like score, backtest chart, technical indicators, recent news, risk notes, feature importance, and signal evidence.

The chosen horizon changes the displayed predicted-change calculation. The LSTM is still trained to classify next-day direction; selecting 1M does not retrain it as a one-month forecasting model.

Backtest

The Backtest page requests historical data for a ticker and displays the backend's strategy curve beside buy-and-hold and a flat naive baseline. It also reports total return, Sharpe ratio, and maximum drawdown. The strategy is currently a momentum-and-RSI rule. It is separate from the LSTM/Random Forest signal ensemble; the chart's Hybrid label should not be interpreted as a historical simulation of the exact Analysis signal.

Models

The Models page describes the major model families and requests a fresh prediction for a ticker. It can show Random Forest and optional XGBoost holdout results, the chronological holdout window, balanced accuracy, and SHAP values where available.

Tuning

The Tuning page saves configuration to the backend. The settings control LSTM lookback length, LSTM units, LSTM epochs, Random Forest tree count, transaction cost, and slippage. The backend validates the values and writes them to backend/tuning_config.json. New model requests and backtests use those settings.

Signals

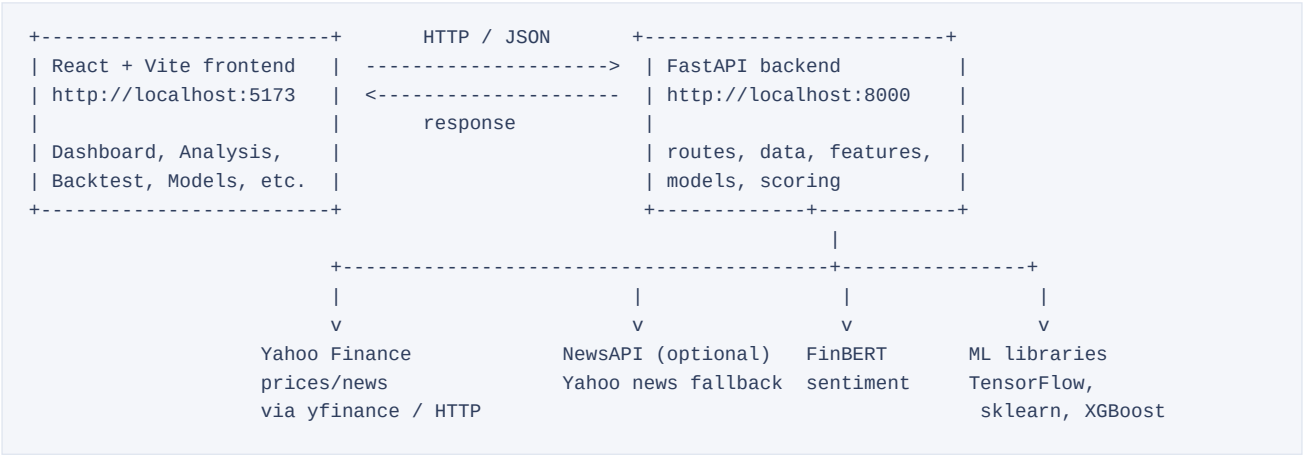
The Signals page lists saved Analysis results in the browser's local storage. A prediction is saved when the user runs Analyze. A newer result for the same ticker and horizon replaces that saved item. Up to 100 items are kept.

Research and Settings

Research summarizes the intended workflow and model families. Settings shows the frontend's local API path and a reminder about backend CORS configuration. The theme selector in the header switches between light and dark mode and remembers the choice in browser storage.

03. System architecture

The frontend and backend are separate programs that run at the same time during development. The browser does not import Python code. It communicates with FastAPI over HTTP using JSON.



The Vite development server proxies requests beginning with /api to port 8000. This lets frontend code call a same-origin path such as /api/predict/AAPL, while Vite forwards the request to FastAPI.

Request lifecycle for an analysis

1. The user enters a ticker and selects a horizon. 2. React calls GET /api/predict/{symbol}?timeframe=.... 3. FastAPI validates the horizon and calls the prediction builder. 4. The backend normalizes the ticker and downloads daily price/volume history. 5. It calculates indicators and creates next-day direction labels for historical rows. 6. It gathers news through configured NewsAPI, Yahoo's search feed, and yfinance fallback. 7. FinBERT scores up to eight headlines, if its model is available. 8. The LSTM and Random Forest components generate direction probabilities. The Random Forest is evaluated on a chronological holdout before being refit on all labeled data for today's probability. 9. Technical evidence and the sentiment score are converted to a normalized component score. The four weighted components produce one combined score. 10. The score is mapped to a signal label. The API returns data, components, explanation, and a backtest. 11. React renders the response and saves the signal to browser local storage.

04. Project files and responsibilities

```
DeepSignal/
|-- backend/
|   |-- main.py                    FastAPI routes, indicators, data, models, scoring
|   |-- requirements.txt           Backend runtime dependencies
|   |-- requirements-dev.txt       Runtime dependencies plus HTTP test client
|   |-- .env.example               Example environment settings
|   |-- tuning_config.json         Created when tuning settings are saved
|   -- tests/
|       -- test_validation.py     Offline automated validation tests
|-- frontend/
|   |-- package.json               Frontend scripts and packages
|   |-- vite.config.js             Vite server and /api proxy
|   -- src/
|       |-- main.jsx               React pages and UI behavior
|       -- styles.css             Layout, components, responsive and theme styles
|-- README.md                      Short setup and project overview
-- docs/
    -- DeepSignal_Project_Guide.md This guide's editable source
```

frontend/node_modules and Python virtual environment folders contain installed packages. They are generated locally and are not source code. The .env file can contain secrets and should remain local; use .env.example as a template.

05. Market data and news

Price history

The download_history function uses yfinance to request adjusted daily data. It asks for Open, High, Low, Close, and Volume (OHLCV), normalizes the date column, removes incomplete rows, and returns a pandas DataFrame. Analysis normally requests three years. History and dashboard routes use shorter selectable periods.

Ticker syntax is supplied to Yahoo Finance. U.S. tickers are commonly plain symbols such as AAPL or MSFT. Indian NSE symbols commonly use the .NS suffix, for example RELIANCE.NS or TCS.NS. Yahoo Finance coverage and symbol conventions can change, and a valid ticker can still return no data.

Prices are fetched when a request runs. The project does not keep a market-price database or cache the downloaded history between requests. The as_of field identifies the last market bar returned by the provider, which may be delayed or reflect the last trading session rather than the current moment.

News collection

NewsAPI is optional. If NEWS_API_KEY is set, the backend first tries its Everything endpoint. When the key is absent, invalid, rate limited, or the response contains too few stories, the backend tries Yahoo's finance search endpoint and then yfinance.Ticker(symbol).news. It normalizes headlines, dates, publisher names, and links, removes duplicate titles, and limits sentiment work to eight headlines.

If every provider fails or returns no articles, the API returns an empty news list. Prediction can continue with a neutral sentiment input. News coverage can be incomplete, late, unrelated to a ticker, or unavailable because of provider/network limits.

06. Feature engineering and indicators

The backend builds technical features from the close and volume columns. The calculations use pandas; TA-Lib is not required.

- SMA 20 / SMA 50: the average closing price over 20 or 50 daily observations. A price above both, with the 20-day average above the 50-day average, is treated as an aligned bullish condition in the technical contribution.
- RSI 14: a momentum oscillator derived from rolling average gains and losses. The implementation uses rolling 14-observation average gain/loss. Low values can indicate recent weakness; high values can indicate strong recent gains. RSI alone does not prove a reversal.
- MACD: the 12-period exponential moving average minus the 26-period exponential moving average. The MACD signal line is a 9-period exponential moving average of MACD.
- Bollinger Bands: the 20-day SMA plus or minus two rolling standard deviations. bb_position estimates where the close lies between the lower and upper bands.
- One-day and five-day return: percentage change in close over one or five trading observations.
- SMA gaps: close divided by an SMA minus one. This expresses distance from the moving average as a proportion.
- Volume change: percentage change in volume from the previous observation.
- Sentiment: one aggregate FinBERT score is attached to the historical rows for the current prediction request.

Rolling calculations need enough history. Early rows contain missing indicator values and are excluded from relevant model training. Infinite values from division are replaced with missing values before model fitting.

07. The model components

LSTM direction model

The LSTM is a recurrent neural network implemented with TensorFlow/Keras. It reads a sequence of recent feature rows and produces one number between zero and one: the model's estimated probability that the next close is higher than the current close. It is a direction classifier, not a model that outputs a future dollar price.

The sequence contains close, SMA 20, SMA 50, RSI, MACD, and Bollinger upper/lower values. The default lookback is 30 observations. The default network has 48 LSTM units, a dropout layer, a 16-unit dense layer, and a sigmoid output. It uses binary cross-entropy and Adam. Early stopping can restore weights from the best validation epoch. Lookback, unit count, and epoch count can be changed on the Tuning page.

The newest 40 training sequences are generally reserved for validation. The current implementation fits the scaler before this split, so information from the later period influences the scaling range. This is one reason to treat reported outputs as experimental and to consider moving all preprocessing inside each training fold in a future evaluation improvement.

If TensorFlow cannot be imported, the LSTM component returns 0.5, a neutral probability. This lets the rest of the pipeline continue, but the result then has no directional contribution from the LSTM.

Random Forest classifier

The Random Forest is a collection of decision trees from scikit-learn. Each tree votes on whether the next close is higher. The backend uses features for one-day and five-day return, SMA gaps, RSI, MACD and signal line, Bollinger position, volume change, sentiment, and an lstm_prob input.

For training rows, lstm_prob is a transformed five-day momentum proxy. For the newest prediction row, this field is replaced by the current LSTM probability. This gives the classifier a historical stand-in for the LSTM feature, but the proxy is not a true out-of-fold LSTM prediction. The distribution mismatch is a methodological limitation.

The default forest has 250 trees, maximum depth 7, minimum leaf size 4, balanced class weights, and fixed random seed 42. It returns a probability for the up class and feature importance values.

XGBoost comparison

If the XGBoost package is available, the backend trains an additional XGBoost classifier, reports holdout accuracy and balanced accuracy, and returns its up probability for model comparison. XGBoost does not currently contribute to the final ensemble score. Its import or runtime errors are caught, so the main result can still be returned without its comparison.

FinBERT news sentiment

FinBERT is a finance-oriented BERT text classifier loaded from Hugging Face Transformers. The default model identifier is ProsusAI/finbert; FINBERT_MODEL can override it. The pipeline is cached once per backend process. On the first use, the model files may need to download and load, which can take time and use disk space and memory.

The backend classifies headlines as positive, negative, or neutral. For each headline it calculates positive score minus negative score. The aggregate sentiment is the average across the processed headlines, normally between -1 and +1. No articles produces 0.0. If FinBERT cannot load or score the articles, headlines are still returned with a null per-article sentiment and the aggregate is neutral.

08. How the final signal is calculated

The current combined score is a weighted average of four values. A component is first expressed on an up-probability-like 0-to-1 scale:

```
combined score =
    0.50 * LSTM probability up
  + 0.30 * Random Forest probability up
  + 0.10 * ((FinBERT sentiment + 1) / 2)
  + 0.10 * technical probability up
```

The technical score starts at zero. The code adds or subtracts points for aligned moving averages, strong five-day momentum, or a specific RSI/MACD reversal condition. The resulting score is clipped to -1 through +1, then remapped to 0 through 1. The technical component therefore contributes at most ten percentage points to the combined score.

The signal thresholds are:

- Combined score at least 0.64: Strong Buy.
- At least 0.54 but below 0.64: Buy.
- Above 0.46 but below 0.54: Hold.
- Above 0.36 and at most 0.46: Sell.
- At most 0.36: Strong Sell.

The Analysis page's Signal Evidence panel shows each component's input, its weight, weighted contribution, and the technical factors. These are the values used by the combination logic. The combined score is an internal model score, not a calibrated probability of making money.

The displayed confidence is calculated from distance from 0.5: 50 + abs(combined score - 0.5) * 100. It is a simple derived display value, not an empirically calibrated confidence interval or a claim that the model is correct that percentage of the time.

The displayed predicted change is also a heuristic. It combines the score's distance from neutral with the five-day return, scales by the square root of the selected horizon, and converts to percent. It is not generated by a price-regression model and should not be treated as a target price.

09. Evaluation and model comparison

The Random Forest and optional XGBoost evaluations use a chronological holdout. The backend sorts rows in historical order, reserves the newest 20 percent (at least 20 rows, while keeping at least 80 earlier rows), fits an evaluation model on the earlier segment, and calculates accuracy and balanced accuracy on the held-out segment. It reports the training/test sample counts and first/last test dates. It then fits a fresh model on all labeled rows to calculate the live prediction.

Accuracy is the share of held-out classifications that match the recorded up/down label. Balanced accuracy averages recall for the up and down classes, which can be more informative when class counts differ. These scores are historical measurements on one holdout window; they do not promise future performance and can vary by ticker and date range.

When fewer than 100 labeled rows are available, the backend omits accuracy rather than inventing a 50 percent score. The live Random Forest probability falls back to 0.5 in that case.

The LSTM's internal validation split is not the same as the Random Forest/XGBoost holdout metrics. The application does not currently provide a full rolling multi-fold evaluation for the LSTM. Also, the whole signal pipeline is not yet evaluated as one end-to-end strategy over repeated untouched periods.

SHAP is optional. When available, TreeExplainer estimates how the Random Forest's features contributed to its output for the current feature vector. It is a local explanation of the forest, not a proof of cause or a full explanation of the LSTM, FinBERT, or final weighted score.

10. Backtesting and its limits

The backtest downloads daily history, computes indicators, and applies a separate rule after a warm-up period. It goes long when the five-observation momentum used by the rule is above 0.6 percent and RSI is below 72. It goes short when momentum is below -0.6 percent and RSI is above 28. Otherwise, it is flat. A transaction-cost-plus-slippage charge is applied when the position changes. The chart also plots buy-and-hold and a flat naive curve.

The backtest's strategy is a simplified momentum/RSI rule. It is not the LSTM/Random Forest/FinBERT ensemble, even though the returned series is named hybrid and the UI labels it Hybrid. Results therefore cannot answer the question, "How would the exact current signal model have performed historically?"

The signal position and same-day return timing should also be treated carefully. A more rigorous backtest should calculate a signal from information available at the end of one bar and apply it to the next bar's return. This helps avoid using a day's closing information to decide a position that earns that same day's return.

Metrics are descriptive:

- Total return: percent change from the starting curve value to the ending value.
- Sharpe ratio: average periodic return divided by return volatility, annualized with the square root of 252.
- Maximum drawdown: largest peak-to-trough percentage decline.

The backtest is a learning aid. It simplifies fills, market impact, trading hours, taxes, currency, corporate actions, survivorship, and data quality.

11. API reference

All endpoints return JSON. The interactive FastAPI documentation is available at http://127.0.0.1:8000/docs after starting the backend.

GET /api/health

Returns a small health response with status ok and a UTC timestamp. It confirms the API process responds; it does not verify Yahoo Finance, FinBERT, or TensorFlow availability.

GET /api/predict/{symbol}?timeframe=1D

Runs the analysis pipeline. timeframe accepts 1D, 1W, 1M, or 3M. The response contains the ticker, date and latest close, signal, trend, predicted-change estimate, sentiment, LSTM and Random Forest probabilities, signal explanation, indicators, news, feature importances, model comparisons, risks, recommendation text, and backtest payload.

GET /api/history/{symbol}?period=1y

Returns OHLCV records. Period accepts 3mo, 6mo, 1y, 2y, or 3y. The Dashboard requests 3mo.

GET /api/backtest/{symbol}

Returns dates, strategy/baseline curves, and metric summaries for the separate momentum/RSI backtest.

GET /api/generalization

Runs the backtest for a built-in small basket of six symbols and reports sector-labeled results or an error per symbol. This endpoint can take time and depends on current provider data availability.

GET /api/config and POST /api/config

GET returns active tuning settings. POST accepts the same fields with bounds. A successful POST writes backend/tuning_config.json and updates the in-process configuration used by subsequent requests.

12. Frontend behavior and local storage

React and React Router render the pages from frontend/src/main.jsx. Recharts renders line and bar plots. The dashboard candlestick chart is drawn as SVG. frontend/src/styles.css contains the responsive layout and both color themes. The theme is stored under deepsignal_theme in local storage.

Saved predictions are stored under deepsignal_signals. The browser stores up to 100 recent items. They are not posted back to the server. This means saved signals are local to one browser profile and can be removed by clearing browser site data or using the Clear History action.

The Vite proxy in frontend/vite.config.js forwards /api requests to http://localhost:8000. For a different backend host, change the proxy or use a deployment-specific frontend configuration, then set the backend's CORS_ORIGINS to the frontend origin.

13. Tuning parameters

The Tuning form sends a POST request to /api/config. Pydantic validates the values before they are applied:

- Lookback: 10 to 120 observations; default 30. Changes the number of rows in an LSTM input window.
- LSTM units: 8 to 256; default 48. Changes the recurrent layer size and model capacity.
- Epochs: 1 to 100; default 6. Sets the maximum number of training passes; early stopping can end training sooner.
- Random Forest trees: 50 to 1000; default 250. Changes the number of trees.
- Transaction cost: 0 to 2 percent; default 0.06 percent.
- Slippage: 0 to 2 percent; default 0.04 percent.

Transaction cost and slippage are combined in the backtest as a charge for a position change. They do not change live predictions. The other four parameters affect model fitting/prediction. More units, trees, lookback, or epochs can make analysis slower and do not guarantee better accuracy.

Configuration is stored in a JSON file beside backend/main.py; it is local to this checkout. Back up or copy that file if you want to carry custom settings to another machine. Keep values within the documented bounds.

14. Installation on another computer

The following assumes the project folder is named DeepSignal. Use two terminals so the API and frontend can remain running together.

Windows PowerShell

1. Install Python with pip and Node.js/npm. Clone or copy the project, then open PowerShell in its root folder (the folder containing backend and frontend).

```
cd C:\path\to\DeepSignal
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r backend\requirements.txt
Copy-Item backend\.env.example backend\.env
```

2. Start the backend in terminal one, from the project root:

```
python -m uvicorn backend.main:app --reload --port 8000
```

3. Start the frontend in terminal two:

```
cd frontend
npm install
npm run dev
```

4. Open http://localhost:5173. The API docs are at http://127.0.0.1:8000/docs.

If PowerShell blocks virtual-environment activation, you can leave it unactivated and call the venv Python directly from the project root. For example, use .venv\Scripts\python.exe -m pip install -r backend\requirements.txt, then .venv\Scripts\python.exe -m uvicorn backend.main:app --reload --port 8000.

macOS or Linux

From the project root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r backend/requirements.txt
cp backend/.env.example backend/.env
```

Start the backend in terminal one, from the project root:

```
python -m uvicorn backend.main:app --reload --port 8000
```

Start the frontend in terminal two:

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173 in a browser.

Optional NewsAPI key

Open backend/.env and set NEWS_API_KEY if you have one. Leave it blank to use Yahoo news fallbacks. Do not put keys in React code, commit the .env file, or share it in screenshots. FINBERT_MODEL defaults to ProsusAI/finbert. CORS_ORIGINS defaults to http://localhost:5173 for local development.

TensorFlow and PyTorch model packages can be large and may require a supported Python/platform combination. If pip reports a wheel or compatibility error, check the Python architecture/version and package support for that system. The requirements file requests modern minimum versions; a virtual environment is strongly recommended.

15. Running the validation suite

The project uses Python's built-in unittest runner. The tests use FastAPI's TestClient and mock external market/news sources, so they do not intentionally call the internet or download model weights.

Install the development requirements from the project root:

```
python -m pip install -r backend\requirements-dev.txt
```

Run the suite:

```
python -m unittest discover -s backend\tests -v
```

On macOS/Linux use forward slashes in the discover path:

```
python -m pip install -r backend/requirements-dev.txt
python -m unittest discover -s backend/tests -v
```

The current suite covers health and prediction route response shape, unsupported timeframe rejection, Buy/Sell/Hold threshold boundaries, persisted tuning configuration and bounds, empty news when providers fail, neutral FinBERT behavior when unavailable, and a neutral LSTM fallback when TensorFlow cannot be imported.

16. Troubleshooting

- Browser says connection refused on port 8000: start the backend and look for Uvicorn running. A blank API page usually means the server process stopped or failed to import.
- Could not import module "main": from the project root use python -m uvicorn backend.main:app --reload --port 8000. From inside backend, use python -m uvicorn main:app --reload --port 8000.
- IndentationError or import traceback: read the final lines of the terminal traceback; correct the Python error before refreshing the page.
- Frontend API error: confirm the backend is listening on port 8000 and the Vite server on 5173. Check the browser's Network panel and the backend terminal.
- No news: this can be a provider response, rate limit, key, or network issue. Prediction should continue with neutral sentiment when there are no headlines.
- First prediction is slow: TensorFlow training and first-time FinBERT loading can be CPU-, memory-, and download-intensive. Wait for the API response and inspect the backend terminal.
- No market history: confirm the ticker format and provider access. For Indian NSE tickers, test the .NS suffix. The backend returns a not-found error when yfinance supplies no bars.
- Tuning save fails: check the Tuning page's validation message and ensure the backend process can write backend/tuning_config.json.
- Port is busy: stop the older process with Ctrl+C or choose another backend port and update the Vite proxy to match.

17. Security, privacy, and operating boundaries

The intended setup is a developer's own machine. The API currently has no login, user accounts, rate limiting, or authorization. Do not expose this development server directly to the public internet. A production deployment would need authentication, authorization for configuration changes, HTTPS, rate limiting, request validation, secret management, logging controls, and a deployment-specific CORS allowlist.

The POST tuning endpoint writes a file on the backend machine. Anyone who can reach an unprotected backend can change the active configuration. The browser's saved signals and theme remain in local browser storage. NewsAPI secrets belong only in the backend .env file. Market and news requests are sent to external providers, so ticker queries and provider traffic leave the local computer.

The project does not execute trades, store brokerage credentials, or persist a user's financial account information. It should be treated as an educational prototype, not a hardened investment service.

18. Current limitations and sensible next steps

The system can be improved without changing the overall architecture:

1. Replace the separate backtest rule with a strictly lagged simulation of the exact ensemble signal, including realistic next-bar execution. 2. Fit data scalers and feature transformations within each chronological training fold to avoid holdout leakage. 3. Generate out-of-fold historical LSTM probabilities for Random Forest training rather than using a five-day momentum proxy. 4. Evaluate more than one chronological window and report sample sizes, class balance, baseline scores, and uncertainty. 5. Calibrate probabilities on a separate validation period before calling them probabilities; rename the current confidence value until calibrated. 6. Cache downloaded OHLCV data and trained models with explicit freshness/version policies to reduce repeated runtime and provider load. 7. Add stronger ticker validation, clearer API error messages, request timeouts, and structured logs that do not include secrets. 8. Add UI loading/error states for Models and other pages, plus tests for build_prediction, the combined-score arithmetic, and backtest timing. 9. Document the exact supported Python/OS/package matrix and add a reproducible lock file or container image.

These are improvement opportunities, not prerequisites for running the local prototype.

19. Glossary

- API: a defined way for one program to request data or actions from another program.
- OHLCV: Open, High, Low, Close, and Volume market-bar fields.
- Feature: a numeric input calculated or collected for a model.
- Label / target: the historical outcome the model is trained to classify. Here it is whether the next close is higher than the current close.
- Probability up: the model's output score for the up class. It is not automatically a calibrated real-world probability.
- Holdout: a later part of historical data kept separate while fitting a model, then used to estimate performance on unseen rows.
- Accuracy: the fraction of classifications that match labels.
- Balanced accuracy: mean recall across the two classes, useful when classes have different sizes.
- Sentiment: a text model's positive/negative tone estimate for a headline.
- LSTM: a recurrent neural network designed to process sequences.
- Random Forest: an ensemble of decision trees that combines their votes.
- XGBoost: a gradient-boosted tree library used here for optional comparison.
- FinBERT: a BERT-family language model adapted for financial text classification.
- Backtest: a simulation that applies a strategy rule to historical data.
- Drawdown: decline from a previous high to a later low in a value curve.
- Slippage: difference between an assumed trade price and a realistic execution price.
- CORS: browser security rules that control which website origins may call an API.
- Local storage: browser-managed key/value storage that stays on one browser profile.

20. Final perspective

DeepSignal joins a local React interface to a Python data and modeling service. It is useful for seeing how data collection, feature engineering, sequence models, tree classifiers, text sentiment, API design, and visualization can fit together. Its most important lesson is also its main caution: a coherent-looking dashboard does not by itself prove predictive skill. Use the displayed data dates, holdout results, component evidence, and backtest assumptions to judge the output, and improve evaluation before treating a signal as actionable.

21. How this guide uses the proposal

The supplied proposal, *A Hybrid Deep Learning and Financial-NLP Framework for Robust Stock Trend Prediction and Trading Signal Generation*, describes a target design. It is not a manifest of features already present in this repository. This guide describes the repository as it exists and marks proposal ideas that are future work. A component is implemented only when source code actually performs it.

The proposal's sequence is: collect and align prices and news; compute technical and text features; forecast direction with LSTM; classify a trade signal with Random Forest and compare boosted trees; explain feature effects; evaluate chronologically with trading costs and baselines; test across stocks; then present results in a dashboard. DeepSignal follows much of this shape, but several implementations are narrower than the proposal or remain incomplete.

Proposal component status

- Python 3.10+: Python is the backend language. Use a Python version compatible with the chosen dependency releases and operating system.
- Yahoo Finance prices: Implemented through yfinance; price history is fetched on request.
- News API: Optional NewsAPI request is implemented with Yahoo Finance news fallback. The app does not archive news history or reconstruct the exact headline set for each training date.
- Kaggle datasets: Not ingested by the code. There is no Kaggle downloader, dataset, or ingestion workflow.
- CSV or SQLite storage: Not implemented for prices or news. Tuning values are stored in JSON; saved signals live in browser local storage.
- Technical indicators and TA-Lib fallback: Indicators use pandas formulas. The backend does not require or call TA-Lib, so there is no runtime TA-Lib fallback.
- LSTM: Implemented as a next-day up/down probability model over recent price and indicator sequences. It does not output a future dollar price.
- FinBERT: Implemented for available headlines. If it cannot load, the system continues with neutral sentiment.
- Random Forest: Implemented as the main tabular classifier, evaluated on a chronological holdout before a full-data refit for current inference.
- XGBoost: Implemented as an optional comparison when it imports and runs. It reports holdout metrics and a probability but does not affect the final combined signal.
- LightGBM: Not implemented and not listed as a runtime dependency.
- SHAP: A Random Forest TreeExplainer attempt is made and failures yield no SHAP values. This does not explain LSTM, FinBERT, or the whole ensemble.
- Walk-forward evaluation: The backtest advances through dates but uses a hand-written momentum/RSI rule, not the model ensemble. RF/XGBoost metrics use one chronological holdout rather than repeated rolling folds.
- Costs and slippage: Included in the rule-based simulation and adjustable on Tuning.
- Baselines: Buy-and-hold is implemented. The naive chart is a constant cash curve, not a trained lag-1 model. The proposal's dedicated LSTM echo baseline is not implemented.
- Multi-stock/sector checks: A fixed six-ticker endpoint evaluates the rule-based backtest, not the full ML classifier across a broad universe.

- Dashboard: The implementation uses React, Vite, and Recharts. Plotly and Streamlit are not in this repository.

22. Deep dive: LSTM model

What the LSTM learns

LSTM means Long Short-Term Memory. It is a recurrent neural network that reads observations in time order and updates an internal state as it goes. Its gates control what past information to retain, update, or forget. This gives it a way to represent patterns across adjacent observations; it does not make market behavior predictable by itself.

DeepSignal's LSTM output is a scalar from zero to one. A value of 0.7 means the network assigns 70% model probability to the next close being higher, given its learned mapping and current input. It does not mean a 70% chance of profit after costs. The model does not produce a target price. The positive label is whether the next row's close is greater than the current row's close.

Exact input and training path

The sequence uses seven columns: close, SMA 20, SMA 50, RSI, MACD, Bollinger upper, and Bollinger lower. The default lookback is 30 observations. A MinMaxScaler scales the columns, and sliding windows are paired with the direction from close at time *t* to close at *t+1*. Rolling indicators create missing values at the beginning; those rows cannot form complete sequences. The final available window is passed to the network for the current probability.

The architecture is one LSTM layer (default 48 units), dropout of 0.2, a 16-unit ReLU dense layer, and one sigmoid output. It uses Adam and binary cross-entropy. The configured epoch count is the maximum; early stopping with patience 2 can stop earlier and restores the best validation weights. Batch size is 32. Lookback, unit count, and epochs are adjustable within validation limits on the Tuning page.

Training requires at least 80 generated windows. Otherwise the component returns 0.5, meaning no directional evidence. If TensorFlow is unavailable, it also returns 0.5. This fallback keeps the API usable but is not a trained result. Model fitting happens again on prediction requests; trained weights are not saved as a reusable production model.

Validation and weaknesses

The code keeps a chronological tail for early stopping validation, about the newest 15% or up to 40 observations. It does not report a separate out-of-sample LSTM accuracy from repeated folds. The scaler is fitted to all available rows before the training/validation boundary is applied, so its min/max values include the later validation period. A rigorous version would fit preprocessing on each training fold only, then apply the frozen transformation to that fold's later rows.

The visible backtest does not retrain this LSTM at each historical boundary, so it cannot establish historical performance of the LSTM. Repeated fitting on live requests also costs time and complicates reproducibility unless data, versions, random seeds, and weights are recorded.

23. Deep dive: Random Forest

What a Random Forest does

A Random Forest trains many decision trees on randomized views of examples and features. A tree makes threshold-based branches, such as whether five-day return is above a value. Trees vote and their votes are averaged into class probabilities. This reduces the instability of a single tree and can represent nonlinear thresholds without scaling every input.

DeepSignal uses the forest as a tabular classifier for whether the next close exceeds today's close. Inputs are one-day return, five-day return, gaps from SMA 20/50, RSI, MACD and its signal, Bollinger-band position, volume change, aggregate news sentiment, and an lstm_prob column.

The historical lstm_prob proxy

For historical Random Forest training rows, lstm_prob is a logistic transformation of five-day return. It is a momentum proxy, not an actual LSTM forecast for each date. For the newest row only, the code substitutes the current LSTM output. This creates a train/inference mismatch. The RF holdout metrics also evaluate on the proxy, not genuine out-of-fold LSTM probabilities.

A stronger approach generates historical LSTM probabilities with walk-forward or out-of-fold fitting, ensuring each row's probability comes from a network that has seen no later data and did not train on that row. The same process must apply inside model evaluation.

Holdout measurement and inference

Rows with missing model inputs or unknown next-day labels are excluded. The newest unlabeled row remains for inference. Below 100 labeled rows, the classifier returns a neutral Hold and omits accuracy. Otherwise it keeps at least 80 older rows for fitting and reserves at least 20 newest rows (or about 20%) as chronological holdout. An evaluation forest is trained on the earlier rows and measured on the later rows. A fresh forest is then fitted on all labeled rows for the current prediction.

Default settings are 250 trees, maximum depth 7, minimum leaf size 4, balanced class weights, and random seed 42. Tree count is tunable; other settings are fixed in source. Impurity-based feature importance is a global summary and can favor features with many possible split points. It is not a causal effect and is not the same as SHAP.

Accuracy is the share of held-out up/down labels correctly predicted. Balanced accuracy averages recall for both classes and helps when class counts differ. Both scores describe one historical window for one ticker and feature setup. They do not include transaction costs, measure profitability, or prove the final ensemble is effective.

24. Deep dive: XGBoost and model comparison

XGBoost is a gradient-boosted tree method. It builds trees sequentially; later trees focus on errors left by earlier trees. Learning rate, depth, regularization, and row/feature sampling help control complexity. It can work well on structured data but can still overfit and must be evaluated chronologically.

DeepSignal imports XGBClassifier inside a guarded optional block. It uses 180 estimators, depth 4, learning rate 0.05, row subsampling 0.85, column subsampling 0.85, log-loss evaluation, and random seed 42. It is evaluated on the same older-train/newer-holdout split as the forest, then refit on all labels for a current up probability. If import, fitting, or inference fails, the backend omits the XGBoost result and continues.

XGBoost is currently a comparison model, not an ensemble voter. Its probability is returned in model_comparison; the final score uses LSTM, Random Forest, FinBERT, and technical evidence. Installing XGBoost therefore does not add its probability to Buy/Sell/Hold. This is one fixed parameter set, not a hyperparameter search. LightGBM appears in the proposal but is neither imported nor used here.

25. Model fusion and signal interpretation

The current combined score is a weighted arithmetic combination: LSTM 50%, Random Forest 30%, FinBERT 10%, and a technical rule score 10%. The model probabilities use a zero-to-one range. FinBERT's signed sentiment from -1 to +1 is converted with (sentiment + 1) / 2. Technical evidence is clipped to -1 through +1 and remapped similarly.

The technical rule checks (1) close/SMA20/SMA50 alignment, (2) five-day momentum outside +/-2%, and (3) an RSI/MACD condition. Those fixed rule points are not a learned model. A neutral component contributes the midpoint 0.5 before weighting.

The combined score maps to Strong Buy at 0.64 or above, Buy from 0.54, Hold between Buy and Sell thresholds, Sell at 0.46 or below, and Strong Sell at 0.36 or below. The evidence panel shows each component, weight, weighted contribution, and technical factors. The score is not a calibrated probability of profit. The displayed confidence is distance from 0.5, not a statistical confidence interval.

The RF's internal classifier label uses its own thresholds, but the prediction builder replaces it with the final weighted ensemble label. Do not confuse the forest-only label with the API's final signal_type.

26. Architecture and data flow in greater detail

Runtime components

- Browser: React and React Router manage pages, form inputs, loading/error states, charts, theme, and local saved signals.
- Development server: Vite serves port 5173 and proxies /api to FastAPI on port 8000.
- API: FastAPI validates inputs, coordinates data retrieval and inference, and returns JSON.
- Market adapter: yfinance retrieves OHLCV history; it is a provider dependency, not a guaranteed exchange feed.
- News adapters: NewsAPI is optional; Yahoo Finance search and yfinance news fill results. Headlines are normalized and deduplicated in memory.
- Features: pandas computes indicators, returns, and the next-row training target.
- Models: TensorFlow/Keras runs LSTM; scikit-learn runs Random Forest; XGBoost is attempted optionally; Transformers runs FinBERT.
- Persistence: backend/tuning_config.json stores tuning values. Browser local storage holds saved signals and theme. Market history, articles, and trained model weights are not persisted.

Request sequence

```
User chooses ticker and horizon
        |
React calls /api/predict/{ticker}?timeframe=...
        |
FastAPI normalizes ticker and retrieves OHLCV history
        +--> NewsAPI (optional) -> Yahoo search -> yfinance news
        |       -> FinBERT scores up to eight headlines, or neutral fallback
        +--> pandas indicators and next-day historical labels
                +--> LSTM sequence model -> probability(next close is up)
                +--> RF holdout metrics, then full-data RF fit
                +--> XGBoost comparison, if available
                +--> technical rule score
        |
Weighted fusion -> threshold -> JSON evidence and summaries
        +--> separate rule-based backtest
        |
React renders result and saves it to browser local storage
```

The browser never needs the NewsAPI secret; the backend contacts the provider. The charting pipeline shares data display with analysis but does not alter model training.

27. Evaluation gaps and honest interpretation

"Walk-forward" can describe different procedures. Here, the backtest advances through dates but uses a fixed rule; RF/XGBoost quality uses one chronological holdout. Neither is a full rolling retraining evaluation of the entire LSTM + RF + sentiment ensemble described in the proposal.

There is a timing issue in the current rule-based backtest: at row i, the position uses indicators through that row's close, then is applied to the return ending at that same row. If an order is placed after observing the close, the return from the prior close to row i has already occurred. The simulation should shift the position to the next return or define an executable decision price/time. Costs are simplified proportional charges when the position changes.

Historical sentiment has another limitation. Current news is fetched at request time and one aggregate score is attached to historical rows. The app does not reconstruct point-in-time news for each date. Historical classifier metrics therefore must not be read as date-correct news evaluation. A rigorous system would timestamp and store articles and only use articles published by each historical decision time.

To close these gaps: define decision time and execution price; shift holdings so a signal earns only later returns; use point-in-time news or exclude sentiment from historical training; fit scalers within each chronological fold; generate out-of-fold LSTM features for the classifier; retrain all ML stages inside repeated expanding/rolling folds; compare to buy-and-hold and a genuine lag-1 direction benchmark; include costs; and report variation across periods and tickers.

28. Setup, reproducibility, and delivery checklist

Windows PowerShell

Run from the project root. These commands make a local environment, install backend packages, create a private environment file, and start FastAPI:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r backend\requirements.txt
Copy-Item backend\.env.example backend\.env
python -m uvicorn backend.main:app --reload --port 8000
```

Open a second PowerShell terminal in the project root:

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173. API health is at http://127.0.0.1:8000/api/health; interactive API documentation is at http://127.0.0.1:8000/docs. Stop each process with Ctrl+C. The .bat launchers are convenience scripts for an already installed local environment.

macOS or Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r backend/requirements.txt
cp backend/.env.example backend/.env
python -m uvicorn backend.main:app --reload --port 8000
```

In a second terminal:

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173. TensorFlow support depends on operating system, architecture, and Python release. If TensorFlow cannot load, the LSTM falls back to 0.5. First use of FinBERT can download model weights and needs internet, disk space, and memory. Provider/package failures can make the displayed output incomplete.

Environment key and reproducibility

backend/.env.example is only a template. Copy it to backend/.env, then optionally add NEWS_API_KEY=.... Keep this file out of frontend variables, screenshots, reports, and version control. Without a key, Yahoo sources are still attempted.

For a reproducible analysis, record ticker syntax, horizon, as_of date, tuning values, Python and package versions, model dependency availability, headlines/provider, and evaluation dates. Providers can change data after a request. This project does not save an immutable run, model checkpoint, market-data snapshot, or historical news snapshot, so a screenshot alone cannot reproduce a prediction exactly.